

Chapter 1

.NET Framework & Automatic Memory Management

ENEL712 Embedded Systems Design — Lecture 18

Summary

Introduction to .NET, the Common Language Runtime, JIT compilation, namespaces, and how the Garbage Collector manages the managed heap (with optimisations and unsafe code).

Topics Covered

- Introduction to .NET
- Core .NET components
- Common Language Runtime (CLR)
- .NET Class Library and data types
- Namespaces
- Automatic memory management (GC)

1.1 Introduction to .NET

- Microsoft framework providing a consistent OOP environment.
- Goals: minimise deployment issues; consistent dev experience across Windows and Web; high performance, security, and integration on industry standards.
- Recent evolution: open-source and cross-platform — ported to Linux/Mac; server-side .NET core open-sourced (e.g., Roslyn).

1.2 Core .NET Components

- Common Language Runtime (CLR): execution engine, runs code, provides services.
- Common Type System (CTS): shared data types across .NET languages.
- .NET Class Library: comprehensive collection of OO types for app development.

1.3 Common Language Runtime (CLR)

CLR handles memory management, thread execution, code-safety verification, compilation, and security.

1.3.1 How the CLR works

- C#/VB.NET → compiled to Common Intermediate Language (CIL/MSIL).
- At runtime, CLR JIT-compiles CIL to native machine code for the host architecture.
- CLR is Microsoft's implementation of the Common Language Infrastructure (CLI) — also implemented by Mono on Linux.

1.3.2 Pros & cons

Feature	Description
Productivity	Devs focus on logic, not low-level memory.
Language interop	Multiple languages compile to one CIL description.
Memory safety	Virtually eliminates leaks and invalid references.
Performance hit	Managed code generally slower than native.
Complexity	Interfacing with legacy native code requires marshalling.

1.4 .NET Class Library and Data Types

- CLS-compliant classes/interfaces for File I/O, strings, collections, DBs, GUIs.
- Integers: Byte (8), Int16 (short), Int32 (int), Int64 (long).
- Floats: Single (float), Double.
- Other: Bool, Char (16-bit Unicode), String, Object (root).

1.5 Namespaces

- Containers grouping related classes — like folders for thousands of classes.
- Hierarchy with dots (e.g., System.IO.Stream).
- using directive imports a namespace at the top of a file.

1.6 Automatic Memory Management

1.6.1 Allocation on the managed heap

- On process start, the runtime reserves a contiguous managed heap.
- NextObjPtr tracks the next free address — allocation is fast (consecutive layout).
- Faster than C-runtime heap, which searches a linked list for a fitting block.

1.6.2 Garbage Collection (GC) algorithm

- Build object list: assume all objects are garbage; trace from Application Roots (globals/statics, locals on stack, CPU registers) to find reachable objects.

- Free and compact: shift non-garbage objects down to overwrite freed space.
- Update pointers: roots and internal references are rewritten to new locations.

1.6.3 Optimisations and unsafe code

- Generational GC: heap split by object age; collections focus on young (Gen 0).
- Pointers in unsafe blocks risk dangling references when GC moves objects.
- GCHandle pins memory so the GC won't move specific objects (used with native interop).

1.7 Use Cases & Applications

- Most desktop applications, web services, and tooling on Windows run on the CLR — knowing how it works helps you debug startup time, deployment size, and memory usage.
- Modern .NET also targets Linux/macOS — useful for Docker images and cloud-native services.
- GCHandle / Marshal lets you safely pass managed memory to native libraries (e.g., USB drivers, vendor SDKs).
- Generational GC means small, short-lived allocations are essentially free; large or long-lived ones (caches, big buffers) deserve more thought.

1.8 Worked Example: Tracing a GC Pass

Imagine a method allocates 1,000 small temporary strings for parsing. After the method returns, all 1,000 are unreachable.

- All 1,000 are placed in Generation 0.
- When Gen 0 fills, GC traces from application roots — the parsing method has returned, so none of the 1,000 are reachable.
- GC frees their memory and compacts Gen 0; survivors (almost none here) get promoted to Gen 1.
- Net effect: cleanup is microseconds and developers never write a `free()` call.

1.9 Code Snippet: Pinning Memory for Native Interop

```
byte[] buf = new byte[1024];
GCHandle h = GCHandle.Alloc(buf, GCHandleType.Pinned);
try
{
    IntPtr ptr = h.AddrOfPinnedObject();
    NativeApi.WriteBytes(ptr, buf.Length); // safe -- GC won't move buf
}
finally
{
    h.Free();
}
```

1.10 Practical Notes

- Don't worry about object allocation cost in normal logic — premature 'optimisation' against the GC usually hurts more than it helps.
- Do worry about retention: long-lived caches that grow without bound, or event handlers you never unsubscribe, become Gen 2 garbage that's expensive to collect.
- Use the dotnet-counters / PerfView tools to inspect Gen 0/1/2 allocation rates in real apps.

Key Definitions

Application Roots

Storage locations (globals/statics, locals, CPU regs) the GC traces from.

CIL

Common Intermediate Language — platform-neutral bytecode that all .NET languages emit.

CLI

Common Language Infrastructure — open spec defining .NET's runtime and code format.

GCHandle

Managed type that pins an object so the GC cannot move it.

Generational GC

Optimisation splitting the heap by object age for faster collections.

JIT Compilation

Runtime translation of CIL to native machine code.

Managed Heap

Contiguous memory region allocated by the runtime for reference types.

Marshalling

Layer/process for interop between managed and native code.

Application Domain

(Legacy .NET Framework) Logical container for code/assemblies; modern .NET uses AssemblyLoadContext instead.

IDisposable

.NET interface for releasing unmanaged resources (files, sockets) deterministically via using or Dispose().

Finalizer

Method run by GC before reclaiming an object; rarely needed when IDisposable is used correctly.

Roslyn

Microsoft's open-source C# / VB.NET compiler platform that exposes parsing and analysis APIs.

Sources

- Lecture 18.pdf